

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	ATHIN SIVA.S	Reg. No.:	192524237
Section / Batch:	2025	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding ; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding ; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 –	Level 3 –	Level 2 –	Level 1 –
----------	-----	-----------	-----------	-----------	-----------

		Exemplary	Proficient	Developing	Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature:	Signature:	Signature:
Date:	Date:	Date:

CO3 AT2-CASE STUDY

Language: Python C C++ Java

QUESTIONS

q1 Web Server Log Analysis (Sorting + Binary Search) A web server processes large log files that must be sorted and searched efficiently
100 marks

1/1 answered

q1 Web Server Log Analysis (Sorting + Binary Search) A web server processes large log files that must be sorted and searched efficiently

100 marks

Analyze how sorting algorithms and Binary Search can be combined. Identify issues such as processing time and large data volume. Apply divide and conquer techniques. Analyze overall complexity. Design a solution and justify its efficiency.

Your Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 // Structure to represent a web server log entry
6 typedef struct {
7     long long timestamp; // Unix timestamp for sorting/searching
8     char ip_address[16];
9     char request[32];
10    int status_code;
11 } LogEntry;
12
13 // Merge function for Merge Sort (Divide and Conquer)
14 void merge(LogEntry arr[], int left, int mid, int right) {
15     int n1 = mid - left + 1;
16     int n2 = right - mid + 1;
```

Input

stdin input

Output

1. Introduction

Web servers generate a large number of log records whenever users access a website. These logs contain useful information such as IP addresses, requested URLs, HTTP status codes, response times, and timestamps. For a busy web server, the number of log records can become very large, making direct searching inefficient. Therefore, efficient algorithms are required to organize and retrieve information quickly.

The proposed Web Server Log Analysis System combines sorting with binary search to efficiently process and search large volumes of web server log data. The main idea is to first arrange the log records according to a searchable field, such as IP address, and then use Binary Search to locate a required record or group of records.

This approach demonstrates the application of Divide and Conquer techniques. Sorting algorithms such as Merge Sort can divide a large dataset into smaller parts, process those parts, and finally combine them. Binary Search also follows the same principle by repeatedly dividing the search range into two halves.

The system is particularly useful when the same log dataset needs to be searched multiple times. Although sorting requires additional processing initially, the sorted data can be searched much faster afterward.

2. Problem Analysis

A web server may produce thousands or millions of log entries every day. Searching such a large dataset using a simple linear search requires checking records one by one.

For example, consider a log containing 1,000,000 records. If a particular IP address needs to be found, Linear Search may need to examine a large portion of the records before finding the required entry. In the worst case, it may have to examine all 1,000,000 records.

The time complexity of Linear Search is $O(n)$, where n represents the number of records.

If many searches are performed, repeatedly using Linear Search becomes expensive. A better solution is to sort the records once and then use Binary Search.

After sorting, Binary Search reduces the search space by half after every comparison. Therefore, its time complexity is $O(\log n)$. For one million records, Binary Search requires only approximately 20 comparisons in the worst case because $\log_2(1,000,000) \approx 20$. This provides a significant improvement when repeated searches are required.

3. Proposed Solution

The proposed system consists of the following major steps:

1. Read or create the web server log records.
2. Store the records in an appropriate data structure.
3. Select a key field for searching, such as IP address.
4. Sort the records based on the selected key.
5. Apply Binary Search to locate a required record.
6. Display the matching log information.
7. Calculate or analyze the overall performance.

A typical log record may contain information such as IP Address, Timestamp, HTTP Method, Requested URL, Status Code, and Response Time.

Example records:

192.168.1.10 | 10:30:15 | GET | /home | 200

192.168.1.15 | 10:31:22 | GET | /login | 200

192.168.1.20 | 10:32:45 | POST | /login | 404

The IP address can be used as the sorting and searching key.

4. Sorting Using Divide and Conquer

Sorting is an important preprocessing step in the proposed system. Binary Search can only work correctly when the data is sorted according to the search key.

A suitable algorithm for large datasets is Merge Sort. Merge Sort follows the Divide and Conquer strategy.

The algorithm divides the original array into two smaller arrays. Each smaller array is recursively divided until individual elements remain. The smaller arrays are then merged in sorted order.

The basic process is: Divide → Solve → Combine.

For example, [40, 10, 30, 20] is divided into [40, 10] and [30, 20]. These are further divided into [40], [10], [30], and [20]. The individual elements are then merged into [10, 40] and [20, 30], and finally into [10, 20, 30, 40].

The time complexity of Merge Sort is $O(n \log n)$ for best, average, and worst cases. This makes Merge Sort suitable for processing large log datasets.

5. Binary Search

Once the log records have been sorted, Binary Search can be used to find a particular IP address or other key.

Binary Search starts by checking the middle element of the sorted array. If the target value is equal to the middle element, the search is successful.

If the target is smaller than the middle element, the right half is discarded and the search continues in the left half. If the target is greater than the middle element, the left half is discarded and the search continues in the right half.

For example, consider 10, 20, 30, 40, 50, 60, 70. To search for 60, Binary Search first checks 40. Since 60 is greater than 40, the left half is discarded. The remaining search space is 50, 60, 70. The middle element is 60, so the required record is found.

The time complexity of Binary Search is Best Case: $O(1)$, Average Case: $O(\log n)$, and Worst Case: $O(\log n)$. This is much faster than Linear Search for large sorted datasets.

6. Overall Algorithm

Step 1: Initialize the web server log records.

Step 2: Store all records in an array or suitable data structure.

Step 3: Select the field that will be used for searching.

Step 4: Sort the records using Merge Sort.

Step 5: Accept the search key from the user.

Step 6: Set the lower and upper boundaries for Binary Search.

Step 7: Calculate the middle position.

Step 8: Compare the search key with the middle record.

Step 9: If both values match, display the corresponding log record.

Step 10: If the search key is smaller, continue searching the left half.

Step 11: If the search key is larger, continue searching the right half.

Step 12: Repeat until the record is found or the search range becomes empty.

Step 13: Display the result and performance information.

7. Complexity Analysis

The overall complexity depends on both sorting and searching.

If there are n log records, Merge Sort requires $O(n \log n)$ time. After sorting, each Binary Search requires $O(\log n)$ time.

Therefore, for one sorting operation followed by m searches, the approximate total complexity is $O(n \log n + m \log n)$. This is highly efficient when many searches are performed on the same dataset.

For example, if there are 1,000,000 records and 10,000 searches, sorting is performed only once. The subsequent searches can be performed using Binary Search rather than repeatedly scanning the complete dataset.

The additional space complexity of Merge Sort is generally $O(n)$ because temporary arrays may be required during the merging process. Binary Search requires $O(1)$ additional space when implemented iteratively.

8. Challenges with Large Data Volume

One major challenge in web server log analysis is the huge amount of data generated over time. A small website may generate thousands of records, while large-scale services can generate millions or billions of records.

Processing large datasets can create problems such as high memory usage, increased processing time, slow searching, large storage requirements, increased input/output operations, and difficulty in real-time analysis.

The proposed sorting and Binary Search approach reduces the searching cost after the initial sorting operation.

For extremely large log files, the data can also be divided into smaller chunks. Each chunk can be processed independently and later merged. This concept is closely related to Divide and Conquer and can also be extended to external sorting techniques when the complete dataset cannot fit into memory.

9. Efficiency and Justification

The main advantage of the proposed approach is that it separates the process into preprocessing and searching.

Sorting may initially take more time than performing a single Linear Search. However, if the same dataset must be searched repeatedly, the initial sorting cost becomes worthwhile.

Linear Search has $O(n)$ search complexity, while Binary Search has $O(\log n)$ search complexity.

For a small number of searches, Linear Search may be acceptable. However, for a large web server log containing millions of records and requiring thousands of searches, Binary Search provides a significant performance advantage.

The solution is therefore most efficient when the dataset is large, multiple searches are required, the data does not change frequently, the search field can be sorted, and fast retrieval is important.

10. Conclusion

The Web Server Log Analysis System using Sorting and Binary Search provides an efficient method for handling large volumes of server log data. The system first organizes the records using an efficient sorting algorithm such as Merge Sort and then applies Binary Search for rapid retrieval.

Merge Sort provides a time complexity of $O(n \log n)$ and uses the Divide and Conquer technique to efficiently organize large datasets. Once the data is sorted, Binary Search can locate records in $O(\log n)$ time.

The overall approach is significantly more efficient than repeatedly using Linear Search, particularly when the same log data must be searched many times. The solution demonstrates how algorithm selection can directly affect the performance of real-world applications.

Therefore, combining Divide and Conquer, efficient sorting, and Binary Search provides a scalable and practical solution for web server log analysis. The approach can further be extended with databases, indexing, parallel processing, real-time log streaming, and distributed storage systems to handle very large-scale web server environments.